

How Software Architects Drive Connected Vehicles

Sören Frey, Lambros Charissis, and Jens Nahm, Daimler TSS

// There are two reasons why successful connected-vehicle projects often assign architecture-related responsibilities to individual experts acting as software architects. First, the experts help effectively manage complexity; second, they act as knowledge multipliers when development must scale up. //



CONNECTING VEHICLES to the Internet of Things is a fascinating field of activity for software architects. Introducing connectivity into vehicles pushes the door wide open for a broad spectrum of potential use cases.^{1,2} For example, consider receiving immediate warnings from other vehicles about traffic jams and black ice on the road ahead.

Or, you might want to remotely unlock your car doors with a smartphone to allow temporary access for

family members. Figure 1 depicts the high-level process flow for controlling the door lock state via mobile devices. A mobile application lets the user issue control commands over a mobile Internet connection to a cloud-based connected-vehicle backend system. For example, the backend system might receive an “unlock doors” command and forward it to the connected car. After the car has received and executed the command, it reports the new door lock state

back to the mobile device via the same communication path.

However, designing a viable solution isn’t as straightforward as this process flow might suggest. The design must take into account a plethora of additional concerns, such as a car manufacturer’s various models, certain regional markets’ technological and legal peculiarities, and security aspects across all the involved systems. For example, the design must often consider secure communication channels and identity and access management. Owing to the large number and variety of those concerns, the architecture design process has an inherent complexity that requires management.

To get started, project teams often deliberately limit the scope. For example, they might restrict the door lock functionality to a specific vehicle model in a pilot phase, using our platform for prototyping telematics services.³ If such pilots prove successful, the need often exists to quickly scale to full production while expanding to new regional markets and covering more of the manufacturer’s vehicles. For example, a pilot that started for a specific car model in Germany might have to be scaled to additional models and standard operation in the overall European market. To enable rapid scaling to multiple project teams, a way is needed to efficiently transfer gained experience among project teams. Managing the connected-vehicle solution space’s complexity and efficiently supporting cross-project communication and knowledge exchange require a corresponding expert role.

We believe that the software architect role is a perfect fit for that expert role, despite the vivid arguments over what software

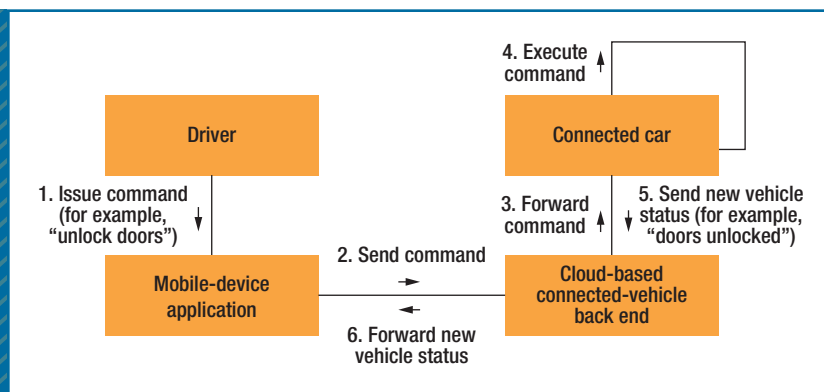


FIGURE 1. The high-level process flow for a remote door lock function. Designing a viable solution isn't as straightforward as this process flow might suggest.

architecture actually is, the blurred profile of the software architect role,⁴ and even the absence of a dedicated software architect role in popular process frameworks such as Scrum. Here, we offer our consolidated observations from five years' experience in the connected-vehicle domain, which covered seven large-scale projects with tens of thousands of customers. In successful projects, the responsibilities regarding software architecture were concentrated in individual experts. These experts, acting as software architects, were crucial to designing complex software systems and scaling the corresponding development. However, these experts' business cards often didn't overtly disclose their software architecture activities.

Architects Manage Complexity

Designing a production-ready solution for the remote door lock function involves several dimensions of complexity and many architectural challenges. Figure 2 gives an overview. A basic remote vehicle function such as door locking should be reusable by many potential users in a variety of applications and use cases,

such as in-car delivery, car sharing, or a concierge service.

Figure 2 depicts examples of the potential users, such as vehicle owners, other drivers, and call centers that need the ability to perform vehicle commands on a vehicle owner's behalf. Whereas a call center agent might use a proprietary application, vehicle drivers use the functionality on devices such as smartphones, tablets, or smart watches. An exposed Web API integrates both the call center case management software and the mobile applications. This context involves architectural considerations such as Web API design, message data formats, and Web security technologies.

Moreover, the remote-door-lock solution must support a variety of vehicles that entail different controller area network⁵ (CAN) signals and network architectures and different telematics control unit (TCU) hardware types and software versions. (CAN is a widespread bus standard for in-vehicle communication; TCU is an embedded system providing a mobile communication interface for vehicles.) One of the most difficult architectural challenges is to provide an abstraction of the many vehicle

idiosyncrasies and offer a single interface to other components relying on remote vehicle functions. This is similar to an anticorruption layer.⁶ In Figure 2, the Virtual Vehicle component accomplishes this task.

The design of an architectural concept for remote vehicle communication can employ two fundamentally different approaches: putting the application logic inside or outside the vehicle. This decision has an enormous architectural impact. When the Virtual Vehicle component sends a remote door lock command to the vehicle, the built-in TCU forwards the command to the CAN, from where the command reaches its target electronic control unit (ECU, an embedded system for controlling a vehicle subsystem).⁷

However, when the application logic is inside the vehicle, the TCU doesn't necessarily send feedback to the Virtual Vehicle component about whether the subsequent door status is what the function call intended. The message might never have reached the vehicle because of a lost connection between the Virtual Vehicle component and the TCU. Alternatively, the TCU might have failed when processing the request, or the door might simply be open and therefore can't be locked. In all cases, the logic of checking the door status after the command has been sent, and retrying the command where required, might be passed through to the user.

This example indicates the need for a technical role that understands the overall system chain, all the way from the user front end to the vehicle's control units. A TCU might have to be updated over the air to not only fix bugs but also add features. If we consider the TCU as a key part of the overall system chain that's under

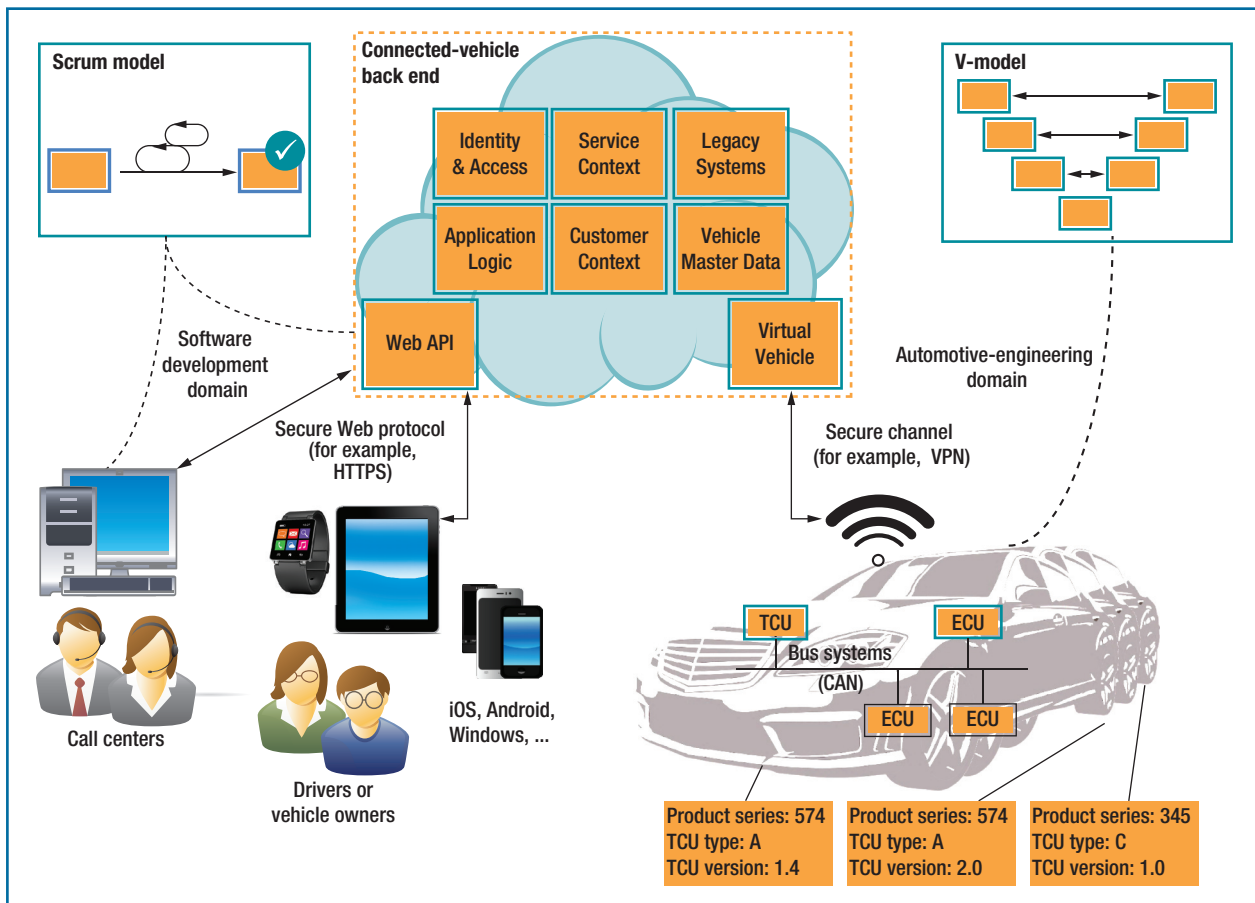


FIGURE 2. The dimensions of complexity in designing a production-ready solution for the remote door lock function. VPN stands for virtual private network, CAN stands for controller area network, TCU stands for telematics control unit, and ECU stands for electronic control unit.

an architect's control, then this role also requires understanding topics that abut the automotive-engineering domain, such as CAN signal encoding and distribution or TCU memory and performance restrictions.

Between the Web API and Virtual Vehicle component is a large system landscape that provides common platform services. Together, these systems constitute the connected-vehicle back end in Figure 2. On one hand, this landscape contains dozens of systems that were developed a long time ago. On the other hand, it includes new back-end systems being developed to

enable vehicle-connectivity use cases, such as the Service Context component in Figure 2.

For building end-to-end solutions, a technical role is needed that also understands the systems and dependencies in the connected-vehicle back end. For example, the back end includes these components:

- The Identity & Access component handles user authentication and authorization.
- The Customer Context and Service Context components hold the information about which

user owns which vehicle and has bought which services.

- The Vehicle Master Data component determines which vehicle can provide which services (for example, remote door locking).

Performing this capability check requires leveraging the vehicle master data. Typically, the system holding this data was in place long before the connected-vehicle features came onto the market and wasn't built to be accessed for thousands of vehicles simultaneously. This is also often the case for other vehicle systems

scattered over several organizational units. So, another architectural challenge we've come across consistently is how to scale and integrate legacy systems from various organizational units that weren't built to be integrated with the connected-vehicle back end. Similar architectural challenges of integrating legacy systems in new environments occur in many other industrial sectors.⁸

Further challenges arise from the various process models that different organizational units and engineering domains use. Typically, the highly safety-sensitive automotive-engineering discipline employs sequential development processes such as the V-model⁹—for example, by following the widespread ISO 26262:2011 standard.¹⁰ Often, the time between the specification phase and final product's completion can be months or years. In contrast, new connected-vehicle back-end services and front ends are often built according to agile, iterative software development methods, such as Scrum or Extreme Programming. Designing a system whose components are built according to completely different process models, release cycles, and engineering mentalities is a major architectural challenge.

From our experience in an enterprise context, it takes a lot of time to

- align with many component representatives regarding overarching technical-integration details,
- request interface extensions according to formal change management processes, or
- reach consensus regarding the deployment details for different staging environments.

We found that concentrating these activities in individual experts who

therefore acted in the software architect role reduced friction. (These experts performed this role formally or informally and sometimes with a varying scope; see the sidebar “The Many Faces of a Software Architect.”) It's also an efficient way to manage the end-to-end design of connected-vehicle functionalities and adequately cope with multiple levels of complexity.

Architects Facilitate Team Scalability

Another aspect of the architect's role deals with knowledge propagation and sharing, which is especially important in a fast-paced domain such as connected-vehicle software. Factors such as the widespread availability of built-in and retrofitted TCUs that let vehicles connect to the Internet, steadily accelerating mobile connection speeds, and users' increasing familiarity with near-ubiquitous connectivity have sparked the automotive industry. Everybody is striving to find well-suited use cases. Some people claim the next big thing is infotainment for drivers and passengers, some claim it's in increasing vehicle utilization through sharing concepts such as car2go (www.car2go.com), and others are following different routes in this vibrant environment.

These product ideas start as a small proof of concept (POC) to attract early as many users as possible. As soon as a POC emits positive signals, companies run a pilot with more users. They then run a scaled pilot with many users before rapidly transitioning to series production. The reference for speed of implementation is often successful Internet companies such as Apple, Google, and Facebook. New agile development teams—say, Scrum teams—must be

created rapidly to enable quick time to market. In this process, ensuring adequate knowledge transfer among the teams is important.

Figure 3 sketches a possible growth path. The two software engineers involved in the POC have a comprehensive overview of the overall architecture and technologies. They act as knowledge multipliers when the new connected-vehicle service becomes a pilot and a Scrum team is staffed for the implementation. The pilot Scrum team (labeled 1 in Figure 3) is in charge of building a more complex solution with deeper integration in the automotive-enterprise-application landscape. For the scaled pilot, new software engineers are added, and a second Scrum team (labeled 2 in Figure 3) is created to handle additional work. Series production (for example, involving additional vehicle models) requires even more development teams (labeled 3, 4, and 5 in Figure 3). The development teams must cooperate and align when implementing an integrated product together.

How should knowledge transfer and alignment among teams be organized? If all the teams use Scrum, a method for scaled Scrum development, such as the Nexus framework,¹¹ is appropriate. The Nexus framework defines a Nexus Integration Team consisting of a dedicated product owner, a Scrum master, and one or more members who might also be in several Scrum teams. The framework matches well with the staged scaling approach in Figure 3. The experts acting as a development team's software architect are a perfect fit for the Nexus Integration Team.

The Nexus Integration Team must manage integration of a connected-vehicle function built from separate



THE MANY FACES OF A SOFTWARE ARCHITECT

In the domain of connected-vehicle software, software experts in an enterprise context act within a wide spectrum of responsibilities and configurations of the software architect role. The following five dimensions broadly characterize how individuals formally and informally carry out this role.

THE JOB TITLE'S RELATION TO THE SOFTWARE ARCHITECT ROLE

The job title adorning an individual's business card might or might not indicate that person's relation to software architecture activities. Besides the straightforward job title of software architect, a wealth of job titles cover a similar scope and include the term "architect," such as solution architect, IT architect, enterprise architect, and system architect. However, the relation to software architecture becomes blurrier when you consider titles that don't include "architect," such as IT consultant, software engineer, and solution developer.

THE AMOUNT OF SOFTWARE ARCHITECTURE ACTIVITIES PERFORMED

From day to day, architects might have to perform software architecture activities that are more or less typical. For example, in our context, solution architects are concerned almost entirely with software-architecture-related activities, such as system decomposition, modeling, and interface definition. In contrast, agile software development frameworks such as Scrum often transfer these activities to whole software development teams. So, individual developers might be involved in such activities in varying degrees.

THE NUMBER AND GRANULARITY OF COMPONENTS IN SCOPE

The software architect role also differs regarding the number and granularity of components with which an architect is concerned. For example, enterprise architects usually design and govern large IT landscapes consisting of many applications. In contrast, some architects might be concerned with only a subset of those applications and their exposed interfaces, to design specific functionalities. The door lock scenario described in the main article falls in this category. On a lower level of abstraction, application architects are typically concerned with designing one or a few applications.

THE PROXIMITY TO PROGRAMMING ACTIVITIES

Software architects perform different amounts of programming. For example, enterprise architects hardly ever program. Furthermore, differences exist among architects with identical job titles. For example, some IT architects might rarely program; instead, they design technical solutions for specific business processes. In contrast, other IT architects might daily contribute source code for one or more applications.

SPECIALIZATION

Software architects frequently specialize. Specializations often occur in specific industrial sectors (such as the automotive industry) and functional domains (such as vehicle connectivity, powertrain optimization, and fabrication). Moreover, specializations frequently focus on specific technologies. This often becomes apparent from the corresponding job titles, such as automotive software architect, Java EE (Java Platform, Enterprise Edition) architect, and cloud architect.

subproducts. To do this, it ensures that each sprint produces an integrated increment. The team members should be software professionals experienced in systems engineering.¹¹

The Nexus Integration Team must also propagate architectural knowledge among Scrum teams on both the interstage level (for example, pilot to scaled pilot) and intrastage

level (for example, scaled pilot to scaled pilot). To do this, it coaches and guides the Scrum teams to acquire, implement, and learn systems-engineering practices and tools.¹¹ It

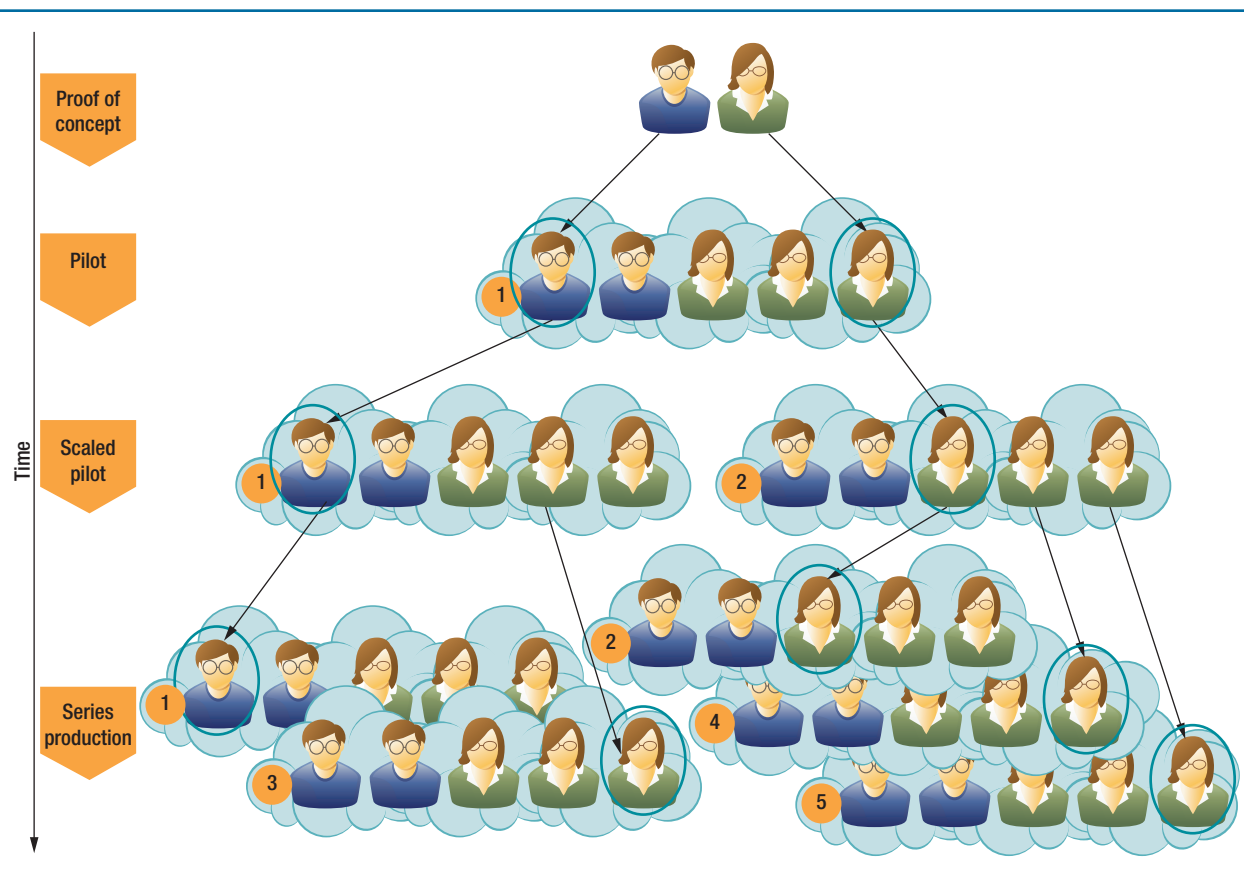


FIGURE 3. Quickly scaling development teams from a few software engineers for a proof of concept to many tens or even hundreds of software engineers is common when companies implement connected-vehicle services. To enable adequate knowledge transfer, software architects (circled in the figure) act as knowledge multipliers and nuclei for new teams. The numbers in the figure indicate the various development teams.

also coaches them on the organizations’ development, infrastructural, or architectural standards.

These tasks can be handled well by individual experts with a strong software architecture background (that might possibly be blended with knowledge from different architectural flavors such as system or solution architecture). These tasks match seamlessly with the tasks software architects typically perform.

So, how should a project split the work among the Scrum teams when it scales up? The most obvious approach would be to build a

front-end team, a back-end team, an embedded-vehicle team, and so forth. This satisfies the specialization found in the labor market and often in organizational structures. However, the teams would lose their independence. We often observed deadlocks and waiting periods when there were more than three or four teams.

Another approach is to build feature teams such that each team develops a specific feature from end to end.¹² Each team must possess the necessary skills, which can be quite challenging. In our experience, each

team must have at least one person with a functional and technical overview and thorough conceptual skills: the software architect. If a team had no architect role, we observed deteriorated code quality and a blurred component structure.

In software development projects, the need still exists for someone with strong conceptual skills, a deep knowledge of the problem domain, and good communications skills—that is, a software architect. Whether we’re talking

about a Scrum team member who focuses on prioritizing technical stories in the backlog, a product owner advisor, an external coach, or many other possible roles with influence, the tasks of an architect remain.

Digitalization as a megatrend reinforces IT's importance. For the automotive sector, the next big trends are autonomous driving and cars as part of the Internet of Things. These trends further increase the underlying IT systems' scope, reach, and complexity. That's why we expect the software architect role to remain a major cornerstone of most complex automotive IT projects in the coming years. 

Acknowledgments

We thank Olaf Zimmermann and the anonymous reviewers for their valuable suggestions and comments on an earlier version of this article.

References

1. M. Alam, "The Top Five Trends for the Connected Car in 2016," *TechCrunch*, 2 Jan. 2016; <http://techcrunch.com/2016/01/02/the-top-five-trends-for-the-connected-car-in-2016>.
2. D. Hardawar, "Buckle Up: The Connected-Car Revolution Is Almost Here," *Engadget*, 13 Jan. 2015; www.engadget.com/2015/01/13/connected-car-revolution.
3. T. Haberle et al., "The Connected Car in the Cloud: A Platform for Prototyping Telematics Services," *IEEE Software*, vol. 32, no. 6, 2015, pp. 11–17.
4. M. Fowler, "Design—Who Needs an Architect?," *IEEE Software*, vol. 20, no. 5, 2003, pp. 11–13.
5. ISO 11898, *Road Vehicles—Controller Area Network (CAN)*, Int'l Org. for Standardization, 2015.
6. E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley Professional, 2004.
7. C. Ebert and C. Jones, "Embedded Software: Facts, Figures, and Future," *Computer*, vol. 42, no. 4, 2009, pp. 42–52.
8. W. Hasselbring, "Information System Integration," *Comm. ACM*, vol. 43, no. 6, 2000, pp. 32–38.
9. J. Weber, "Vehicle Development Projects—an Overview," *Automotive Development Processes: Processes for Successful Customer Oriented Vehicle Development*, Springer, 2009, pp. 1–15.
10. ISO 26262:2011, *Road Vehicles—Functional Safety*, Int'l Org. for Standardization, 2011.
11. K. Schwaber, *Nexus Guide v1.1*; www.scrum.org/Resources/The-Nexus-Guide.
12. C. Larman and B. Vodde, *Scaling Lean & Agile Development: Thinking and Organizational Tools for Large-Scale Scrum*, Addison-Wesley Professional, 2008.

ABOUT THE AUTHORS



SÖREN FREY is a senior IT architect of connected-vehicle software at Daimler TSS. His research interests are connected vehicles, cloud computing, software architectures, and machine learning. Frey received a PhD in computer science from Kiel University. He's a member of the IEEE Computer Society, ACM, and the German Informatics Society. Contact him at soeren.frey@daimler.com.



LAMBROS CHARISSIS is an IT architect of connected-vehicle software at Daimler TSS. His research interests are connected vehicles, software architectures, and cloud computing. Charissis received a master's in computer science from the University of Stuttgart. Contact him at lambros.charissis@daimler.com.



JENS NAHM is the manager of the Enterprise Architecture Team at Daimler TSS. His research interests are connected vehicles, software architectures, and cloud computing. Nahm received a Dipl. Inform. in computer science from the University of Stuttgart. Contact him at jens.nahm@daimler.com.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.